# /autopilot

Toggle autonomous autopilot mode (/autopilot on|off|status)

## Overview

`/autopilot` runs the agent autonomously with permission auto-approval. In autopilot the agent keeps working across turns without waiting for confirmation, and it stops itself when it decides the task is complete - the `task_complete` completion signal is documented in the `on` help text - or when a budget limit is reached.

## Syntax

```
/autopilot on [--max-tokens N] [--max-time N]
/autopilot off
/autopilot status
/autopilot help
```

## Options / Subcommands

| Form | Description |
|---|---|
| `/autopilot on` | Enables autopilot with permission auto-approval |
| `--max-tokens N` | Stop after N tokens are consumed (flag may appear anywhere after `on`) |
| `--max-time N` | Stop after N seconds (flag may appear anywhere after `on`) |
| `/autopilot off` | Clears the enabled state, both budgets, the start time, and any pending autopilot continuation |
| `/autopilot status` | Prints `ON` with the elapsed seconds, or `OFF` |
| `/autopilot help` | Prints the help table |

Both flags are optional and can be given in any position. Caveat: the flag parser silently ignores unknown tokens, so a typo in a flag name is not reported - the run simply starts without that budget applied.

## Examples

`/autopilot on`

Starts an autopilot run with no explicit budget.

`/autopilot on --max-tokens 100000`

Runs autonomously until roughly 100k tokens have been consumed.

`/autopilot on --max-time 1800`

Stops the run after 30 minutes.

`/autopilot on --max-time 600 --max-tokens 50000`

Flags may be given in any order after `on`.

`/autopilot status`

While running, prints `ON` with the number of elapsed seconds; otherwise `OFF`.

`/autopilot off`

Ends autopilot mode and clears the budgets and timers.

## Output

- `on`: a confirmation message describing the mode, the optional budgets, and the `task_complete` signal the agent uses to end the run.
- `status`: `ON` plus elapsed seconds, or `OFF`.
- `off`: a confirmation message.

## Related

- `/yolo` - persistently bypass permission prompts instead of per-run approval
- `/loop` - goal-driven loop with verification gates and stop conditions
- `/task` - task tracking used alongside autonomous runs